

Reviewer Pack — Minimal Rank of Tropical Division Algebras vs SAT Proof Leng...

Entry 97e6bf05f9a9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 10:26:45 UTC

Minimal Rank of Tropical Division Algebras vs SAT Proof Length

Entry ID: 97e6bf05f9a9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 10:26:45 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For every instance of a SAT problem with n variables, the minimal rank of the division algebra associated with its tropical semi-ring is upper-bounded by a function of n .’, ‘This function $f(n)$ satisfies $f(n) = O(\log^2(n))$.’, ‘The minimal rank in question refers to the smallest dimension of a vector space over the field of two elements that can be generated from the tropical semi-ring associated with the SAT problem.’]

Rationale (proposer’s reasoning):

[‘Tropical division algebras have been previously used to encode arithmetic operations and are related to the geometry of valuation rings. Their ranks could provide insights into the complexity of solving SAT problems by capturing structural properties that resolution proofs exploit.’, ‘Minimal rank serves as a measure of algebraic complexity, potentially revealing nontrivial relationships between tropical algebra and combinatorial logic.’, ‘Previous work in tropical geometry has explored connections with other complexity measures, suggesting a fertile ground for further investigation.’]

Taxonomy category: TROPICAL_DIVISION_ALGEBRAS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d2a8e5aebab828d0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all seeds 1 to 30 for SAT instances with n variables ($n \leq 40$), the ratio of the minimal rank to $\log^2(n)$ is less than or equal to a constant C for at least 80% of the samples. The conjecture is falsified if this ratio exceeds C for any seed, or if less than 80% of the samples meet this condition.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND resolution proof complexity AND minimal rank division algebra - SAT problem minimal rank tropical semi-ring resolution proof length $O(\log^2(n))$ - upper bound on minimal rank for tropical division algebras in terms of SAT variables

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(1, 2 * n) for _ in range(random.randint(1, n))]
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def tropical_semi_ring(clauses):
        n = len(clauses[0])
        semi_ring = [[-math.inf] * n for _ in range(n)]
        for clause in clauses:
            for lit in clause:
                if abs(lit) > n:
                    continue
                i = abs(lit) - 1
                semi_ring[lit - 1][i] = max(semi_ring[lit - 1][i], 1)
                semi_ring[i][lit - 1] = max(semi_ring[i][lit - 1], 1)
        return semi_ring

    def minimal_rank(semi_ring):
```

```

n = len(semi_ring)
rank = 0
for i in range(n):
    if any(x > 0 for x in semi_ring[i]):
        rank += 1
return rank

def resolution_length(clauses):
    # Simplified version of resolution length calculation
    return sum(len(clause) for clause in clauses)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5): # Test each n with 5 instances
        clauses = generate_sat_instance(n)
        semi_ring = tropical_semi_ring(clauses)
        rank = minimal_rank(semi_ring)
        length = resolution_length(clauses)
        results.append((n, rank, length))

if not results:
    return {
        "metric_name": "minimal_rank_over_log2_n",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

total_rank = sum(rank for _, rank, _ in results)
total_length = sum(length for _, _, length in results)
mean_rank = Fraction(total_rank, len(results))
mean_length = Fraction(total_length, len(results))
ratio = mean_rank / (mean_length ** 2)

support_fraction = sum(1 for _, rank, _ in results if rank <= math.log(n) ** 2) / len(results)

return {
    "metric_name": "minimal_rank_over_log2_n",
    "metric_value": float(ratio),
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"Ratio {ratio} exceeds log^2(n)"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

```

```

total_rank = sum(result["metric_value"] * result["instances_tested"] for result in results if
total_instances = sum(result["instances_tested"] for result in results)
mean_ratio = Fraction(total_rank, total_instances)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Ratio exceeds log^2(n)\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE Reason=No valid data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c339d273.py", line 95, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c339d273.py", line 60, in run_trial
    semi_ring = tropical_semi_ring(clauses)
                ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c339d273.py", line 38, in tropical_semi_ring
    semi_ring[lit - 1][i] = max(semi_ring[lit - 1][i], 1)
                        ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture according to the pre-registered support and falsification conditions. | next: Investigate the cause of the crash in the test code and attempt to run the test again to gather the necessary data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14403
2	preregistration	ollama_remote	glm4:latest	0	0	9507
3	novelty	ollama_remote	glm4:latest	0	0	8489
4	novelty	ollama_remote	glm4:latest	0	0	9043
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12542
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9587
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9151
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11645
9	judge	ollama_remote	glm4:latest	0	0	11690

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96057 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/97e6bf05f9a9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/97e6bf05f9a9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/97e6bf05f9a9.tar.gz` (if generated)